

Introduzione all'Orchestrazione di macchine virtuali, servizi e container mediante Kubernetes

Per dettagli e approfondimenti, vedere la meravigliosa serie di articoli di Rinor Maloku: "Kubernetes and everything else". (<https://rinormaloku.com/series/kubernetes-and-everything-else/>).

Vedere anche <https://kubernetes.io/docs/home/>

Sommario

1. Kubernetes Concepts.
2. Kubernetes Architecture
3. Prodotti per Orchestrazione di Container basati su Kubernetes.
4. Capire le esercitazioni in aula - Organizzazione delle macchine virtuali sul pc del docente.
5. Installazione di minikube e creazione di cluster.

1. Kubernetes Concepts

Kubernetes è una architettura software con una struttura client-server che esegue su un **cluster** (cioè un gruppo di host) in cui dispiega la sua applicazione. L'interfacciamento dell'amministratore di kubernetes avviene tipicamente mediante un client, ad esempio il client `kubectl`.

NODES: Gli host del cluster vengono detti nodi.

PODS: Rispetto all'architettura di swarm, kubernetes organizza maggiormente l'applicazione definendo delle unità, dette **pods**, composte da uno o più container che hanno necessità di lavorare strettamente tra di loro come se fossero tutti in uno stesso host virtuale. I container di uno stesso pods, infatti, sono realizzati per poter:

- **condividere uno stesso indirizzo IP,**
- **avere lo stesso spazio delle porte di protocollo (cioè due container in uno stesso pod non possono attestarsi sulla stessa porta di protocollo),**
- **comunicare tra di loro mediante il localhost,**
- **interagire tra di loro mediante le classiche Inter Process Communications (IPC).**

Invece, due container contenuti in pods diversi per comunicare tra di loro devono specificare l'indirizzo IP del pods a cui appartiene l'altro container.

Ogni pods è contenuto in un solo host fisico del cluster, ma in uno stesso host fisico possono essere contenuti più pods, ciascuno dei quali risulterà dotato di un proprio indirizzo IP.

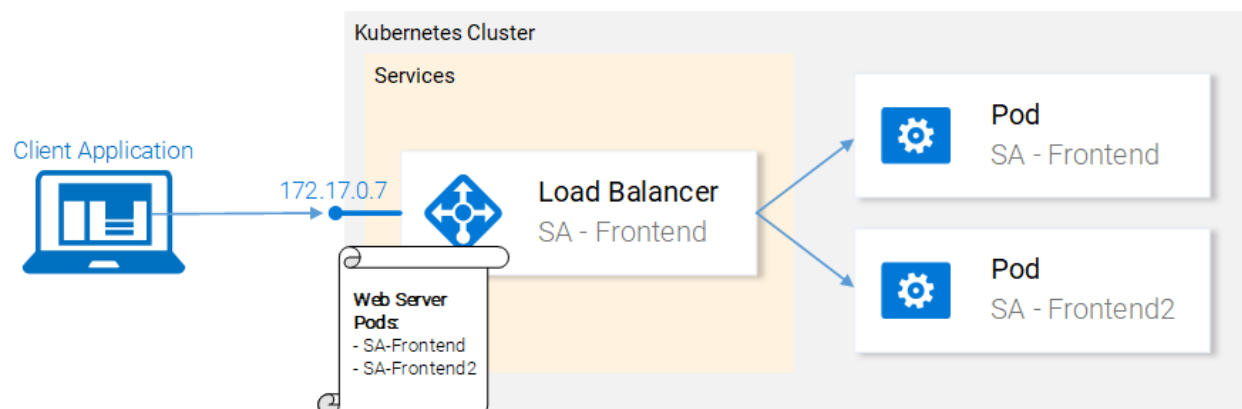
LABELS: Ciascun pods può essere classificato mediante una label, cioè una stringa dotata di un nome e di un valore. Tutti i pods a cui è stata assegnata una stringa con stesso nome e stesso valore possono essere considerati equivalenti per fornire una funzionalità. Ad esempio, alcuni pods potrebbero essere classificati mediante la label di nome role e con valore production (“role=production”). Quella label potrà essere utilizzata da kubernetes per individuare i pods a cui richiedere un servizio. Questi pods equivalenti funzioneranno come delle repliche per fornire quella funzionalità.

SELECTORS: La ricerca di una risorsa può essere effettuata specificando un selettore, cioè specificando come filtro una label ed il suo valore richiesto. Ad esempio, si potrebbero cercare i pods classificati mediante la label “role=production”.

DEPLOYMENTS: Può essere utile dispiegare più istanze di uno stesso pod, in modo da creare delle repliche su cui distribuire il carico di lavoro. Perciò, quando kubernetes dispiega un pod, per cui idealmente possono essere previste delle repliche, chiama deployments questo gruppo di repliche di pod. Ovviamente, qui emerge l'esigenza di selezionare questi pods e di bilanciare il carico tra loro.

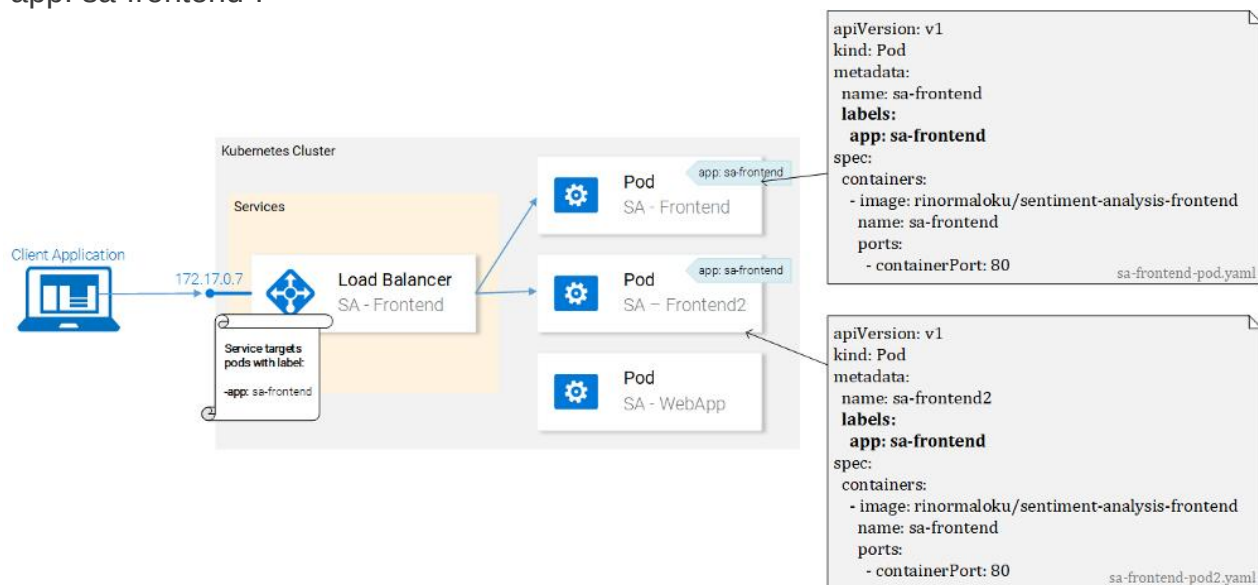
SERVICES: Infine, kubernetes definisce il concetto di servizio, ovvero un insieme di pods solitamente individuati da un selettore. Il servizio comprende il modo per accedere ai pods, di solito l'indirizzo IP del pod o il nome con cui il DNS mappa quel pod. Il servizio tipicamente include anche la politica con cui viene effettuato il bilanciamento di carico delle richieste tra i diversi pods di quello stesso servizio. La politica di bilanciamento di carico può essere gestita e implementata da kubernetes stesso ma, in caso si voglia ottenere qualche effetto specifico, può essere utilizzato un servizio di bilanciamento di carico esterno.

Di solito, quindi, ogni gruppo di pods definiti da uno stesso selettore è protetto da un servizio che regola l'accesso ai pods di quel servizio.



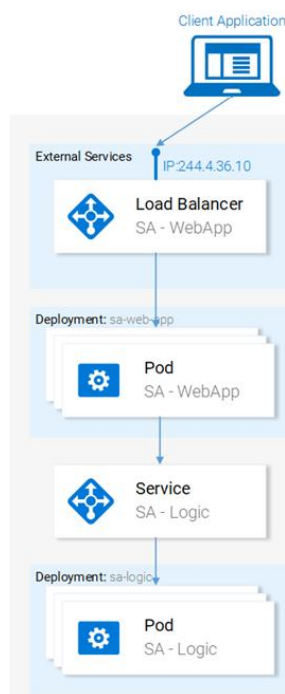
Operativamente, quando si dispiega una applicazione in un cluster, prima si istanziano i singoli Pods (o le repliche), creando il deployment, poi davanti a ciascun gruppo di pods si istanzia il servizio che regola l'accesso ai pods.

Per dispiegare ciascun gruppo di pods e ciascun servizio si utilizza un file che descrive le proprietà di quel pod o di quel servizio. Un caso semplice è rappresentato nella seguente figura, in cui si ipotizza che due pods, denominati sa-frontend ed sa-frontend2 siano descritti dai file indicati sulla destra. Si noti come il nome di ciascun pod (metadata: name:) sia diverso, ma il selettore (metadata: labels: ...) sia lo stesso "app: sa-frontend".



E' possibile e comune anche una situazione, come quella rappresentata nella figura qui sulla destra, in cui un servizio acceduto dal client (Load balancer SA-WebApp) inoltra la richiesta verso un gruppo di pods (Pos SA-WebApp) che svolgono una parte del servizio.

Ciascuno di questi pods però, per completare il servizio ha necessità di ottenere un servizio da un altro gruppo di pods (Pod SA-Logic) e per effettuare la richiesta si rivolge al Servizio (Service SA-Logic) che si occupa di inoltrare la richiesta ad uno dei suoi pods, seguendo una qualche politica di bilanciamento di carico.



AUTOSCALING: Tecnica utilizzata nel cloud computing per incrementare e/o ridurre in modo dinamico l'ammontare di risorse computazionali messe a disposizione da una server farm.

Quando si fa uso del cloud computing, si acquistano solamente le risorse necessarie per l'uso che si deve fare con i servizi che vengono messi a disposizione.

Il calcolo delle risorse necessarie non è semplice, perché non è prevedibile a priori.

Infatti, il traffico può subire dei picchi al variare dell'orario e del giorno in questione.

Queste variazioni potrebbero non essere supportati dalle risorse che sono state acquistate dal cloud provider e di conseguenza i servizi esposti potrebbero provocare una perdita di prestazioni e dare un disservizio agli utenti che vorrebbero utilizzarlo.

Grazie all'autoscaling, Le aziende che fanno uso di cloud privati possono spegnere e/o mandare in standby le macchine non necessarie per poter risparmiare sui costi dell'elettricità.

Le aziende che fanno uso di cloud pubblici possono ridurre i loro costi pagando solamente le risorse utilizzate quando necessarie.

Le soluzioni di Autoscaling (come quella fornite da Amazon Web Services) consente anche di sostituire le istanze definite *unhealthy*, ossia che hanno tempi di risposta medi troppo elevati, per evitare fallimenti dei servizi offerti.

Autoscaling in Kubernetes: Kubernetes offre tre strumenti per effettuare autoscaling:

- **Horizontal Pod Autoscaler**
- **Vertical Pod Autoscaler**
- **Cluster Autoscaler**

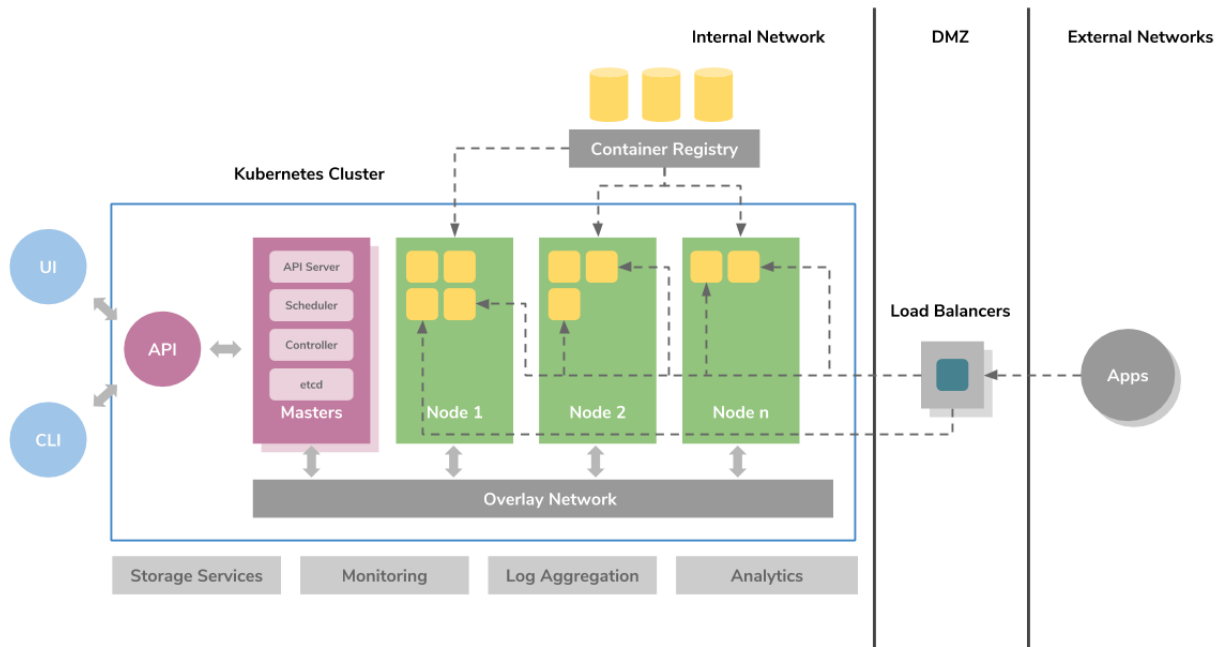
L'**Horizontal Pod Autoscaler (HPA)** è il meccanismo che consente di scalare automaticamente **il numero di repliche di un Pod** sia in incremento che in riduzione sulla base dell'osservazione di determinate metriche (uso di CPU, RAM, ...).

Il **Vertical Pod Autoscaler (VPA)** è il meccanismo che consente di incrementare le risorse (CPU e Memoria) richieste da un Pod al Nodo su cui è dispiegato.

Cluster Autoscaler è il meccanismo che consente di aumentare o diminuire il numero di nodi di un cluster su cui eseguire i pods.

2. Kubernetes Architecture

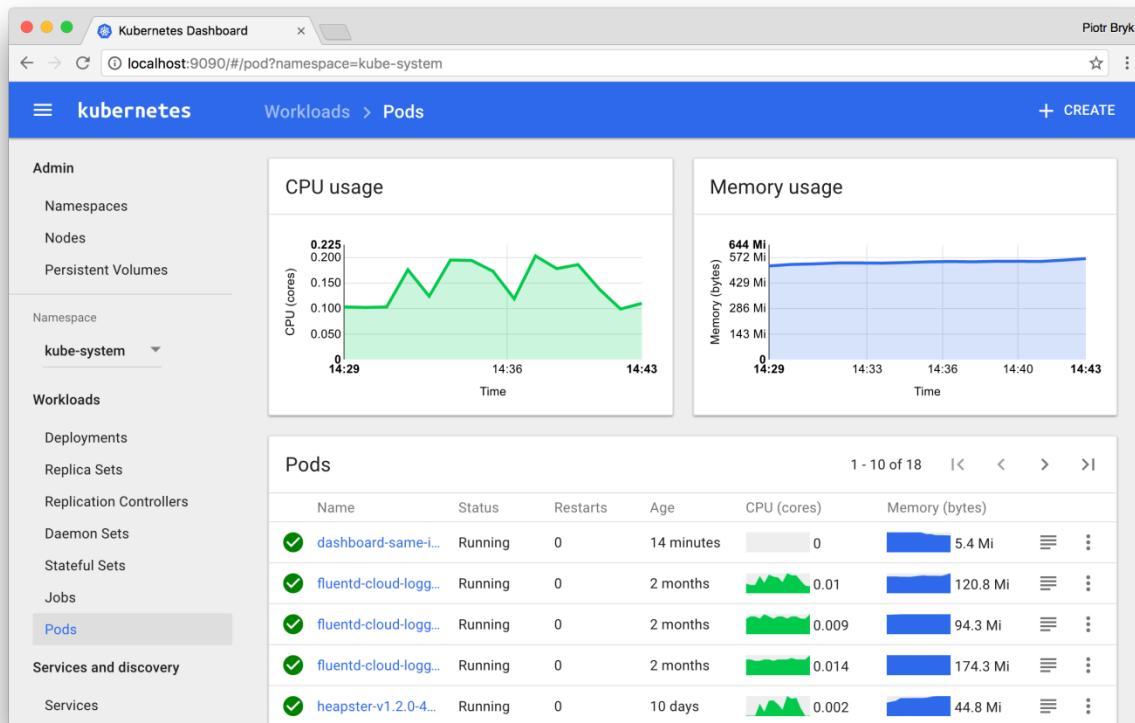
L'architettura di kubernetes prevede che i nodi del cluster siano connessi da una rete di tipo overlay e che tra i nodi del cluster ci sia un nodo che svolga il ruolo di master.



3. Prodotti per Orchestrazione di Container basati su Kubernetes.

Una infrastruttura per gestire servizi realizzati mediante container ed orchestrati utilizzando Kubernetes, può essere dispiegata in **cloud** oppure **on premise**. In questo secondo caso potrebbe essere utilizzato **un cluster di nodi Linux** oppure, in modo minimale e a scopo di apprendimento, **un singolo nodo**.

In tutti i casi, kubernetes mette a disposizione una dashboard di controllo con cui monitorare i servizi.



Nel caso cloud, come infrastrutture per dispiegare applicazioni esistono gli approcci di google (GKE, Google Kubernetes Engine), in cui kubernetes è già disponibile come servizio. In alternativa, gli approcci di Amazon AWS e Microsoft Azure prevedono l'installazione di kubernetes in una macchina virtuale (per ora, ma probabilmente a breve verranno offerti come servizi integrati o forse esistono già).

Ancora per il caso cloud, esistono poi alcuni ambienti di apprendimento per kubernetes, detti kubernetes playground, in cui mediante un browser si possono mettere in esecuzione delle applicazioni di prova su un cluster esistente in cloud. Tra questi ambienti ricordiamo katakoda (<https://www.katacoda.com/courses/kubernetes/playground>), che mette a disposizione un cluster di 2 nodi.

Per il caso on premise, che usa cluster di nodi dell'utente che vuole dispiegare l'applicazione, si potrebbero usare distribuzioni Linux qualunque su cui installare i necessari applicativi. Esistono però anche distribuzioni Linux che sono progettate proprio per svolgere il ruolo di nodi di cluster per kubernetes: RancherOS (distribuzione open source <https://rancher.com/rancher-os/>), Tectonic (prodotto commerciale, <https://coreos.com/tectonic>).

Nel caso invece si voglia realizzare una infrastruttura minimale in un unico nodo, si potrà utilizzare Minikube, che è un applicativo che genera una macchina virtuale in cui emulare un cluster e mettere in esecuzione i diversi pods ed i loro container. Minikube fornisce

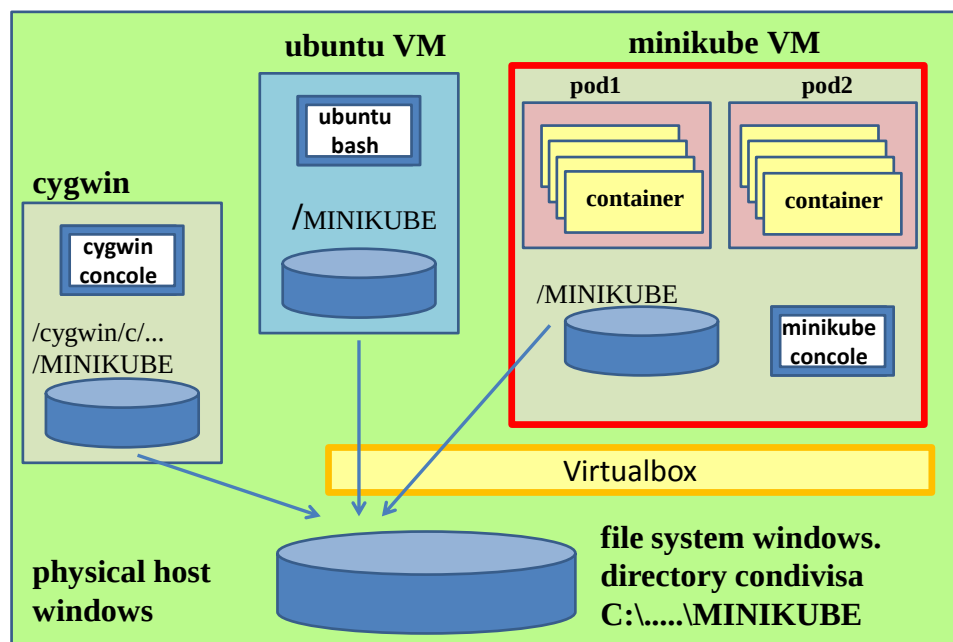
strumenti a riga di comando per gestire i cluster. Assieme a minikube si installa **kubect1**, che fornisce l'interfaccia di kubernetes necessaria all'orchestrazione dei servizi dispiegati dall'utente nel cluster di minikube. Anche qui è disponibile la **dashboard di kubernetes**.

Minikube può operare su macchine Linux, Windows e Mac.

4. Capire le esercitazioni in aula - Organizzazione delle macchine virtuali sul pc del docente.

Nell'esempio che segue, installo minikube nella mia macchina windows, sfruttando l'hypervisor Virtualbix, già installato.

VMs in macchina docente



Poiché minikube è una macchina virtuale minimale in cui eseguiranno i pods e container della mia applicazione gestiti da kubernetes, la macchina contiene pochi degli strumenti di sviluppo di cui avrei invece bisogno per realizzare le mie applicazioni. Non esiste nemmeno apt. Anziché installare dentro minikube le mie applicazioni opero diversamente, condividendo una directory nel filesystem dell'host fisico windows e utilizzandolo dall'interno delle macchine virtuali che lo mappano. In particolare:

1) Nel filesystem fisico di windows colloco i sorgenti delle mie applicazioni da eseguire nella directory

C:\DIDATTICA\SystemIntegration\DISPENSE\MINIKUBE\

2) Dentro windows, ho installato una macchina virtuale **Ubuntu** 18.04 LTS dentro virtualbox e gli faccio vedere quella directory di windows come se fosse la directory /media/MINIKUBE/

Dall'interno di questa macchina virtuale lancio i comandi docker build e quant'altro serve per creare le immagini dei container, salvandoli nella directory condivisa.

3) Sempre nel sistema windows ho installato gli applicativi a riga di comando minikube.exe e kubectl.exe. Però mi viene comodo utilizzare questi comandi come se mi trovassi in ambiente Linux.

4) Per poter avere una interfaccia Linux in ambiente Windows ho installato l'emulatore di Linux su Windows, cygwin. Cygwin vede la directory delle mie applicazioni come se fosse:

`/cygdrive/c/DIDATTICA/SystemsIntegration/DISPENSE/MINIKUBE/`

Dall'interno di cygwin vedo le immagini create dei container e le posso utilizzare, e posso anche mettere in esecuzione gli applicativi windows a riga di comando minikebe.exe e kubectl.exe. Perciò, da cygwin per prima cosa faccio partire il cluster minikube con il comando installato in windows

`minikube start`

e successivamente metterò in esecuzione le mie applicazioni invocando kubectl con i necessari parametri ed i file di configurazione.

5) Quando lancerò il comando `minikube.exe start` verrà creata una macchina virtuale minikube che ospita i container di cubernetes e poi ospiterà i pods delle mie applicazioni. In questa macchina virtuale, mappo la directory /MINIKUBE/ dentro la directory di windows delle mie applicazioni. La macchina virtuale fornisce una console da cui sono fruibili i comandi di docker installati all'interno della macchina virtuale. Il comando docker interno a questa amacchina virtuale è quello che viene utilizzato per operare sui container quando io invoco i comandi minikube.exe e soprattutto kubectl.exe dalla console cygwin.

5. Installazione di minikube.

Minikube deve essere installato direttamente su una macchina fisica. Se viene installato su Windows è indispensabile però che prima venga installato un hypervisor, consigliati Virtualbox o Hyper-V. Se invece voglio installare minikube su una machina Linux potrei fare a meno dell'hypervisor purché installi minikube specificando un flag **--vm-driver=none**. In Linux posso comunque utilizzare un hypervisor, consigliati Virtualbox o Kvm. In ambito Mac, consigliati Virtualbox, VMware Fusion, o HyperKit.

Prima di installare minikube occorre comunque sempre installare kubectl.

Qui di seguito, i passi per l'installazione di Minikube:

- a) Verificare che l'host fisico supporti la virtualizzazione.
- b) Installare uno degli hypervisor consigliati.
- c) Installare kubectl.
- d) Installare minikube.
- e) Eseguire minikube creando il cluster di nodi virtuali.
- f) Customizzazione della macchina virtuale minikube.
- g) Usare la dashboard di kubernetes.

In ambiente Windows i passi da effettuare si svolgono seguendo le seguenti modalità:

- a) **Verificare che l'host fisico supporti la virtualizzazione.**

To check if virtualization is supported on Windows 8 and above, run the following command on your Windows terminal or command prompt.

```
systeminfo
```

If you see the following output, virtualization is supported on Windows.

```
Hyper-V Requirements:      VM Monitor Mode Extensions: Yes
                          Virtualization Enabled In Firmware: Yes
                          Second Level Address Translation: Yes
                          Data Execution Prevention Available: Yes
```

- b) **Installare uno degli hypervisor consigliati.**

Per installare Virtualbox vi arrangiate.

- c) **Installare kubectl direttamente usando i file binari.**

Install kubectl binary with curl on Windows.

To find out the latest stable version (for example, for scripting), take a look at <https://storage.googleapis.com/kubernetes-release/release/stable.txt>.

1. Download the latest release v1.14.1 from this link

<https://storage.googleapis.com/kubernetes-release/release/v1.14.1/bin/windows/amd64/kubectl.exe>

Or if you have curl installed, use this command:

```
curl -LO https://storage.googleapis.com/kubernetes-release/release/v1.14.1/bin/windows/amd64/kubectl.exe
```

Userò minikube da cygwin, quindi lo copierò nella directory di cygwin in /usr/local/bin

```
chmod 777 /cygdrive/c/Users/vittorio/Downloads/kubectl.exe
```

```
mv /cygdrive/c/Users/vittorio/Downloads/kubectl.exe /usr/local/bin/
```

2. Add the binary in to your PATH.

3. Test to ensure the version you installed is up-to-date:

```
kubectl version
```

Verifying kubectl configuration

In order for kubectl to find and access a Kubernetes cluster, it needs a kubeconfig file, which is created automatically when you create a cluster using kube-up.sh or successfully deploy a Minikube cluster. By default, kubectl configuration is located at ~/.kube/config.

Check that kubectl is properly configured by getting the cluster state:

```
kubectl cluster-info
```

If you see a URL response, kubectl is correctly configured to access your cluster.

If you see a message similar to the following, kubectl is not configured correctly or is not able to connect to a Kubernetes cluster.

The connection to the server <server-name:port> was refused - did you specify the right host or port?

For example, if you are intending to run a Kubernetes cluster on your laptop (locally), you will need a tool like Minikube to be installed first and then re-run the commands stated above.

If kubectl cluster-info returns the url response but you can't access your cluster, to check whether it is configured properly, use:

```
kubectl cluster-info dump
```

d) **Installare minikube.**

To install Minikube manually on Windows, download minikube-windows-amd64, at <https://github.com/kubernetes/minikube/releases/latest> save it in /usr/local/bin and rename it to minikube.exe, and add it to your path.

```
wget
```

```
https://github.com/kubernetes/minikube/releases/download/v1.0.1/minikube-windows-amd64.exe
```

```
mv minikube-windows-amd64.exe /usr/local/bin/
```

```
mv /usr/local/bin/minikube-windows-amd64.exe /usr/local/bin/minikube.exe
```

```
chmod 777 /usr/local/bin/minikube.exe
```

If you have previously installed minikube, and run:

```
minikube start
```

And this command returns an error:

```
machine does not exist
```

You need to wipe the configuration files.

Cleanup everything to start fresh:

```
rm -rf ~/.minikubeCleanup everything to start fresh.
```

e) **Eseguire minikube creando il cluster di nodi virtuali.**

Nell'emulatore cygwin, creare un cluster di nodi virtuali (un solo nodo in realtà). Eseguire il comando

```
minikube start
```

Viene effettuato il download della macchina virtuale e fatto partire il cluster.

```
"minikube" stopped.
vittorio@alien ~
$ minikube start
minikube v1.0.1 on windows (amd64)
$   Downloading Kubernetes v1.14.1 images in the background ...
i   Tip: Use 'minikube start -p <name>' to create a new cluster, or '
minikube delete' to delete this one.
:   Restarting existing virtualbox VM for "minikube" ...
:   Waiting for SSH access ...
:   "minikube" IP address is 192.168.99.100
:   Configuring Docker as the container runtime ...
:   Version of container runtime is 18.06.3-ce
:   Waiting for image downloads to complete ...
:   Preparing Kubernetes environment ...
:   Pulling images required by Kubernetes v1.14.1 ...
:   Relaunching Kubernetes v1.14.1 using kubeadm ...
:   Waiting for pods: apiserver proxy etcd scheduler controller dns
:   Updating kube-proxy configuration ...
:   Verifying component health .....
+   kubectl is now configured to use "minikube"
=   Done! Thank you for using minikube!
```

Notare l'indirizzo IP del cluster: 192.168.99.100

Notare anche alcuni pods che sono messi in esecuzione: apiserver proxy etcd scheduler controller dns

Nella shell che compare all'interno della macchina virtuale minikube potete dare ordini. Quando compare una shell che chiede login, loggarsi come root, non chiede password.

Invece, nella shell cygwin di windows, potete ordinare la terminazione del cluster locale, mediante il comando:

```
minikube stop
```

Questo comando non distrugge il cluster ma ne termina l'esecuzione.

Invece, se utilizzate il comando

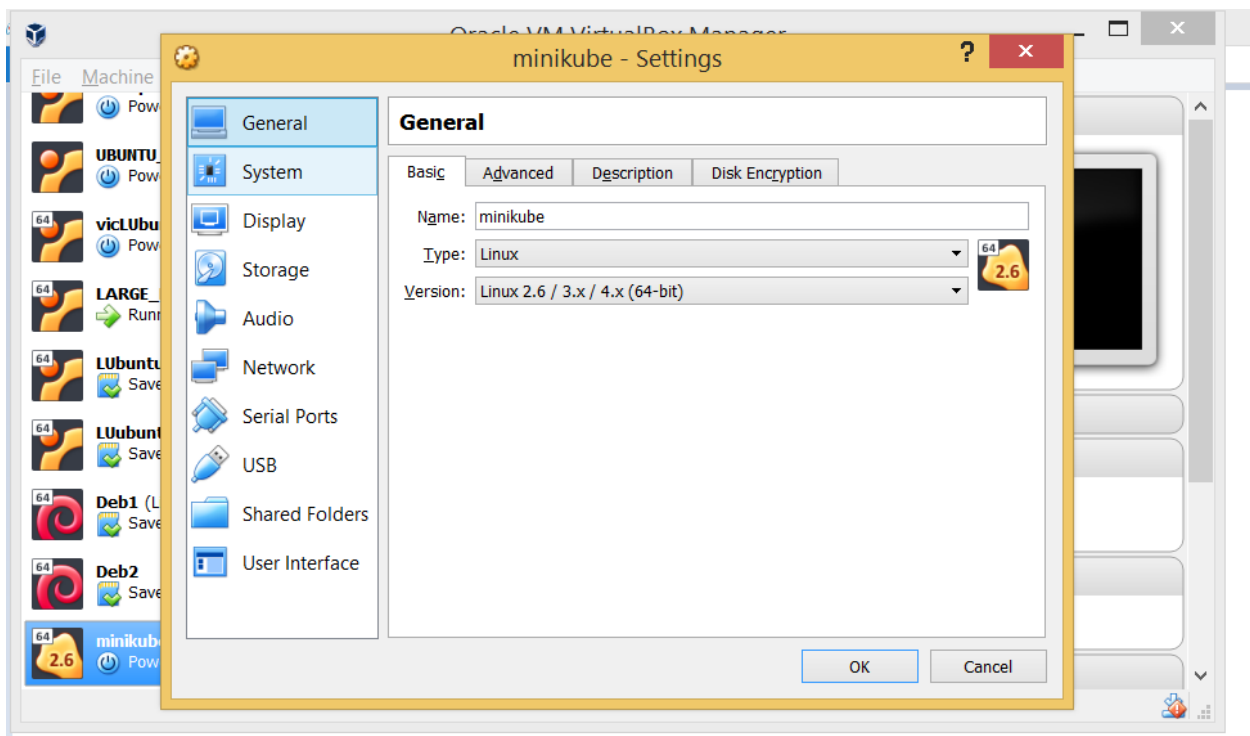
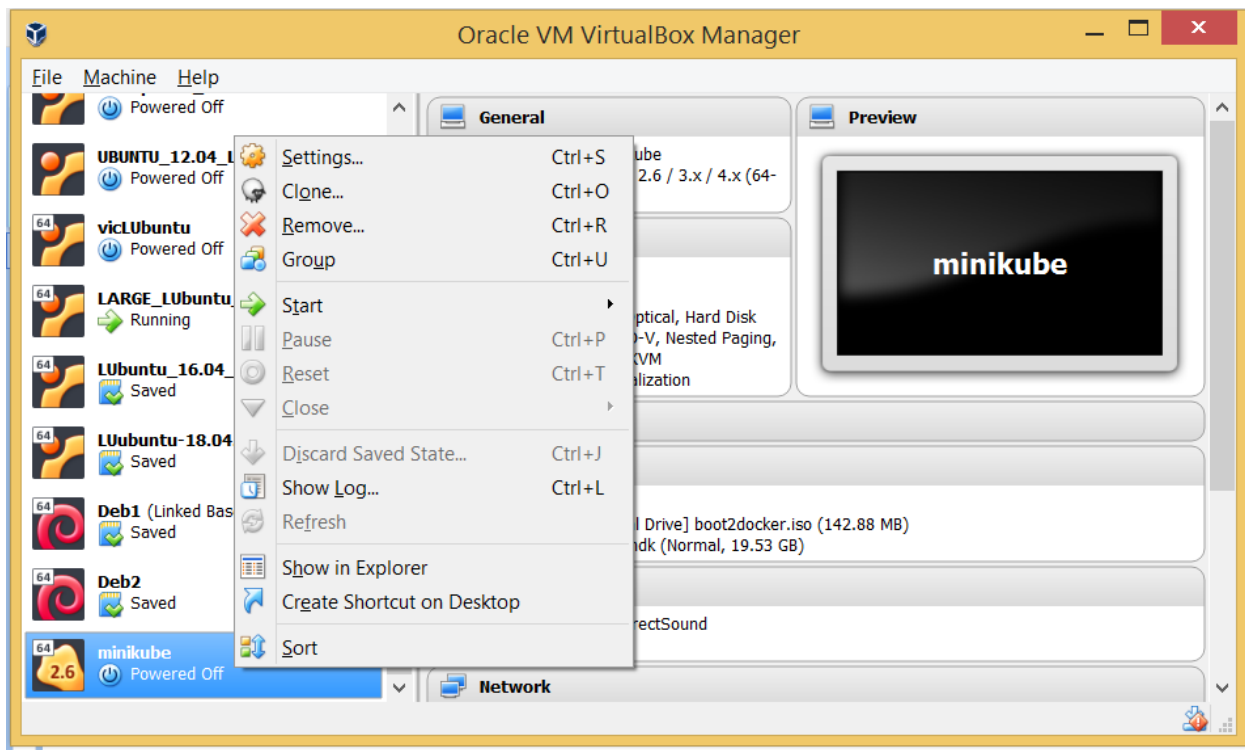
```
minikube delete
```

eliminate la macchina virtuale minikube e tutte le sue impostazioni.

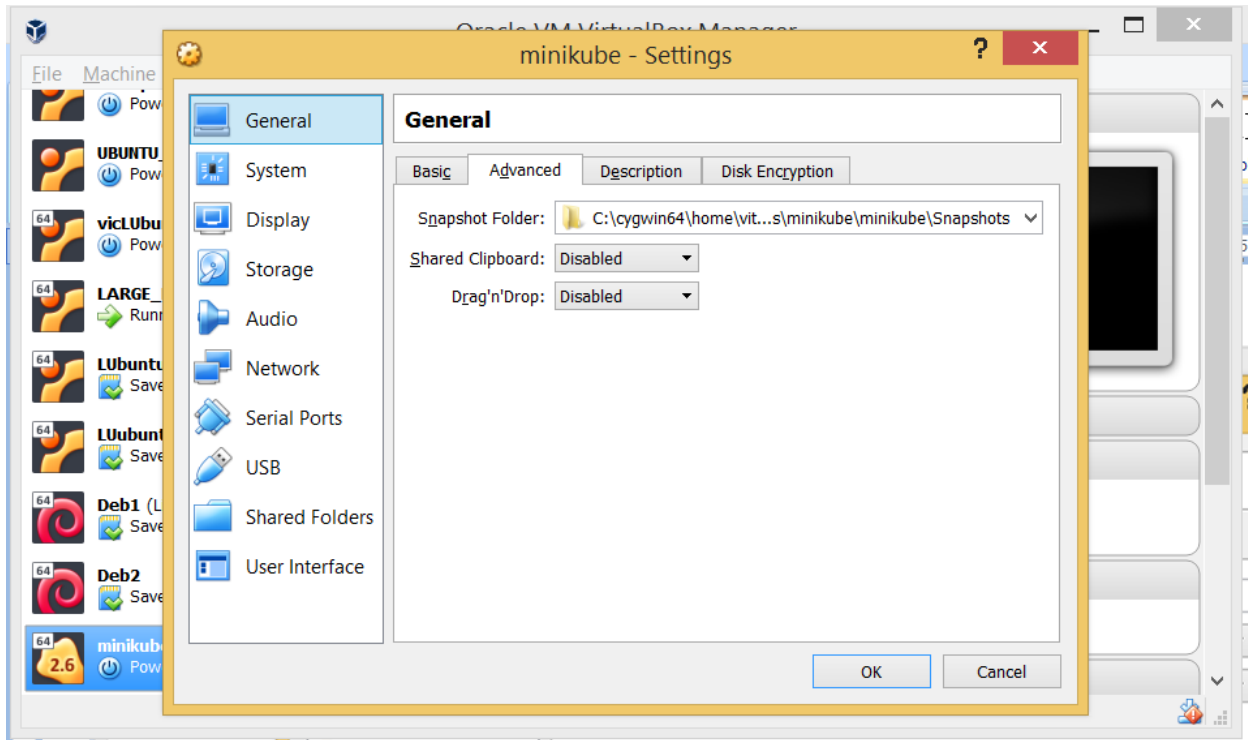
f) Customizzazione della macchina virtuale minikube.

CUSTOMIZZARE LA MACCHINA VIRTUALE minikube

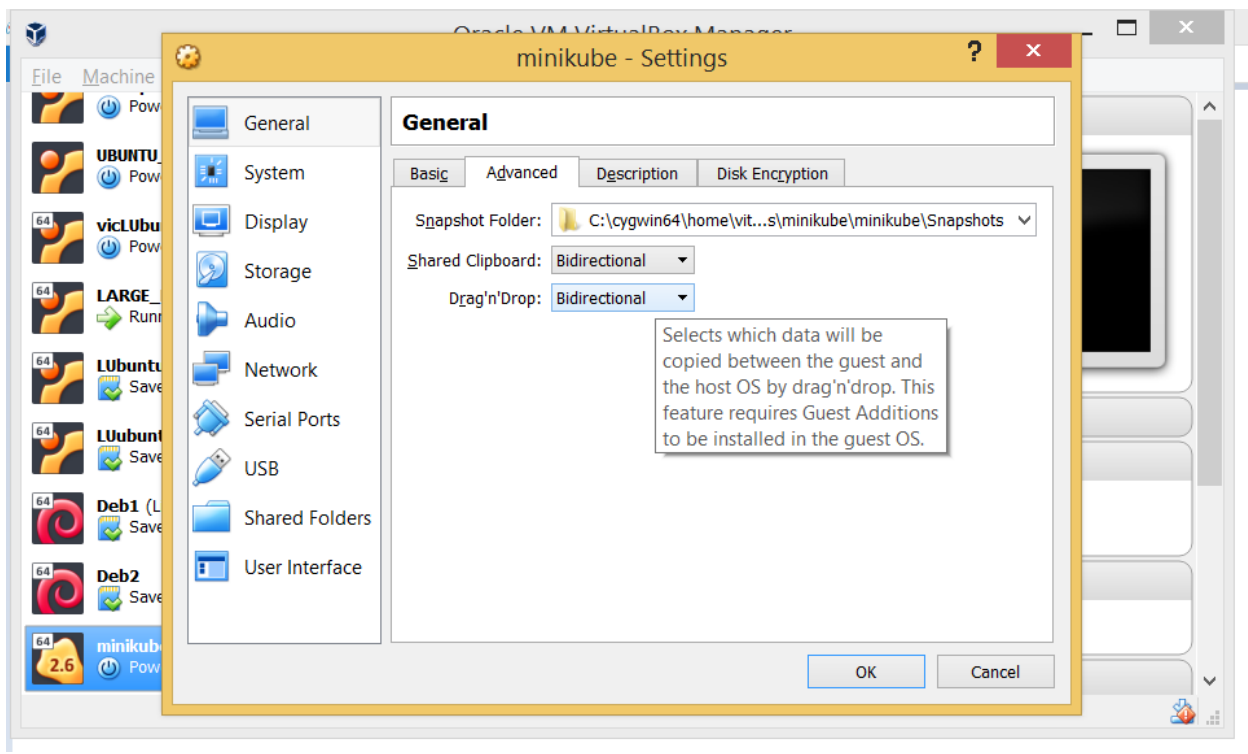
selezionare la macchina "minikube", pulsante destro del mouse, menù Settings, si apre una nuova finestra.



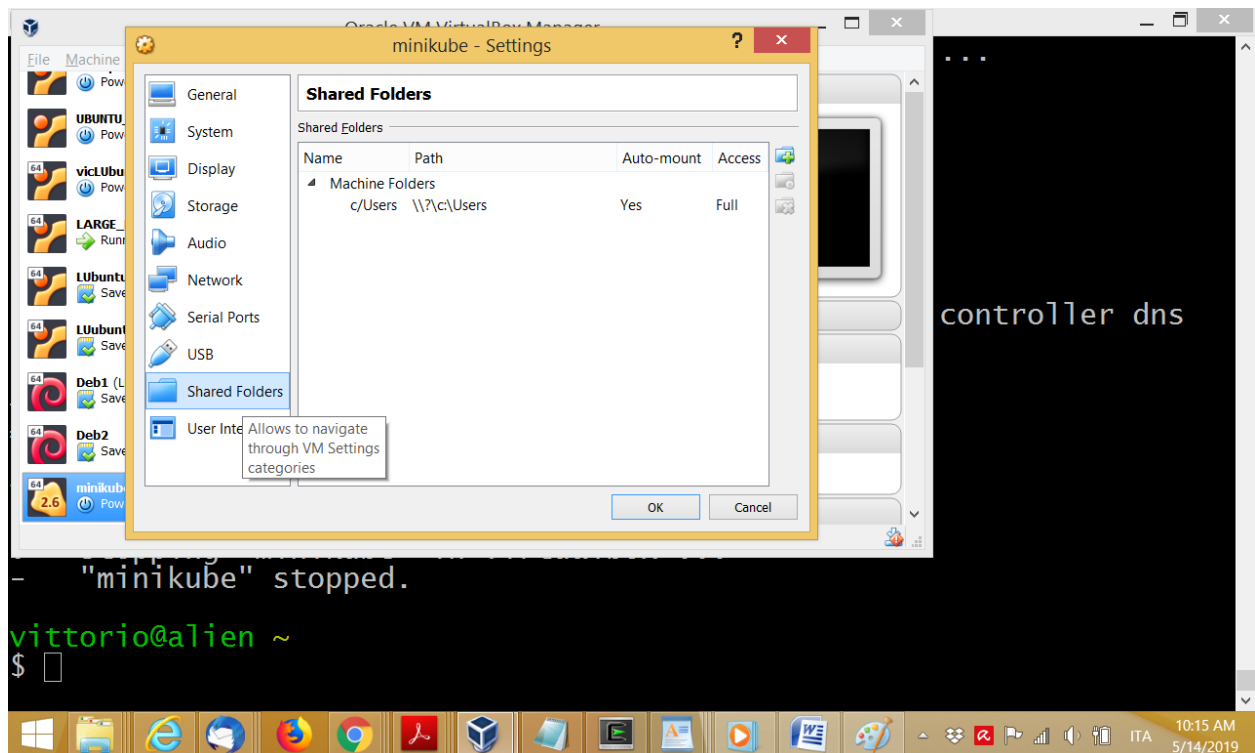
Selezionare "General" in alto a sinistra, selezionare il Tab "Advanced"



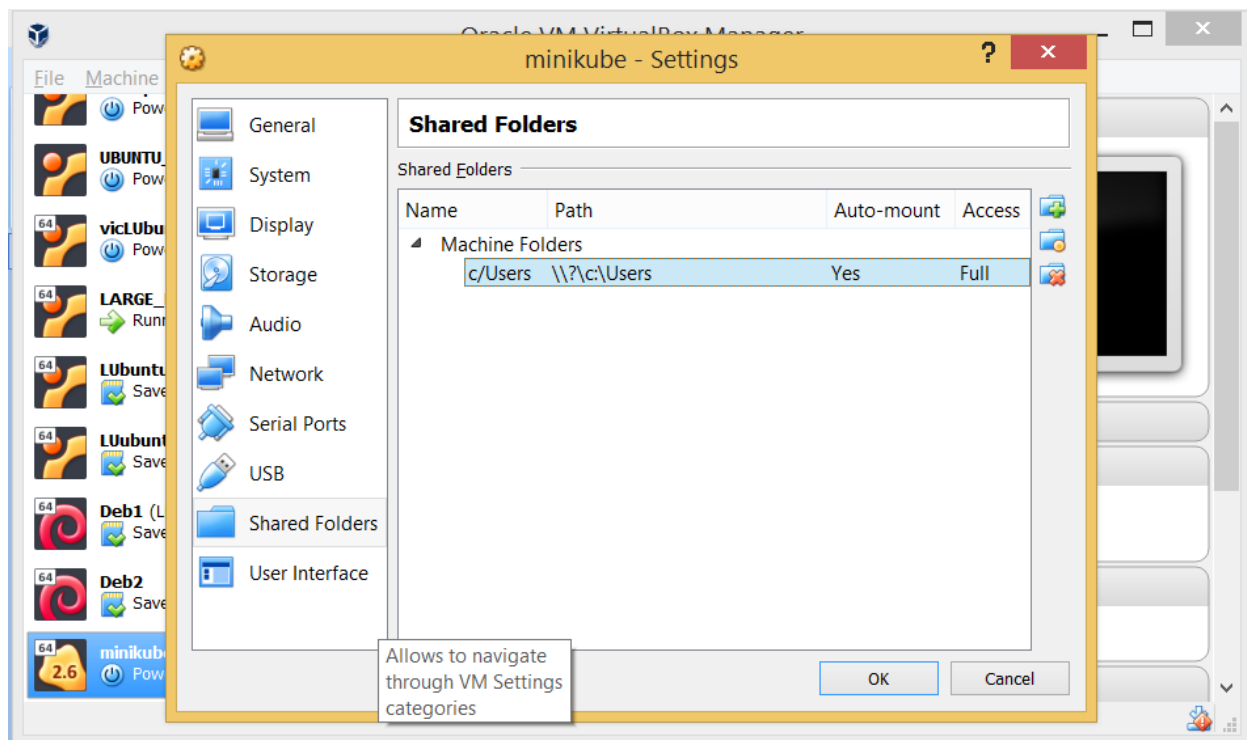
SELEZIONARE PER ABILITARE Shared clipboard bydirectional and drag and drop.



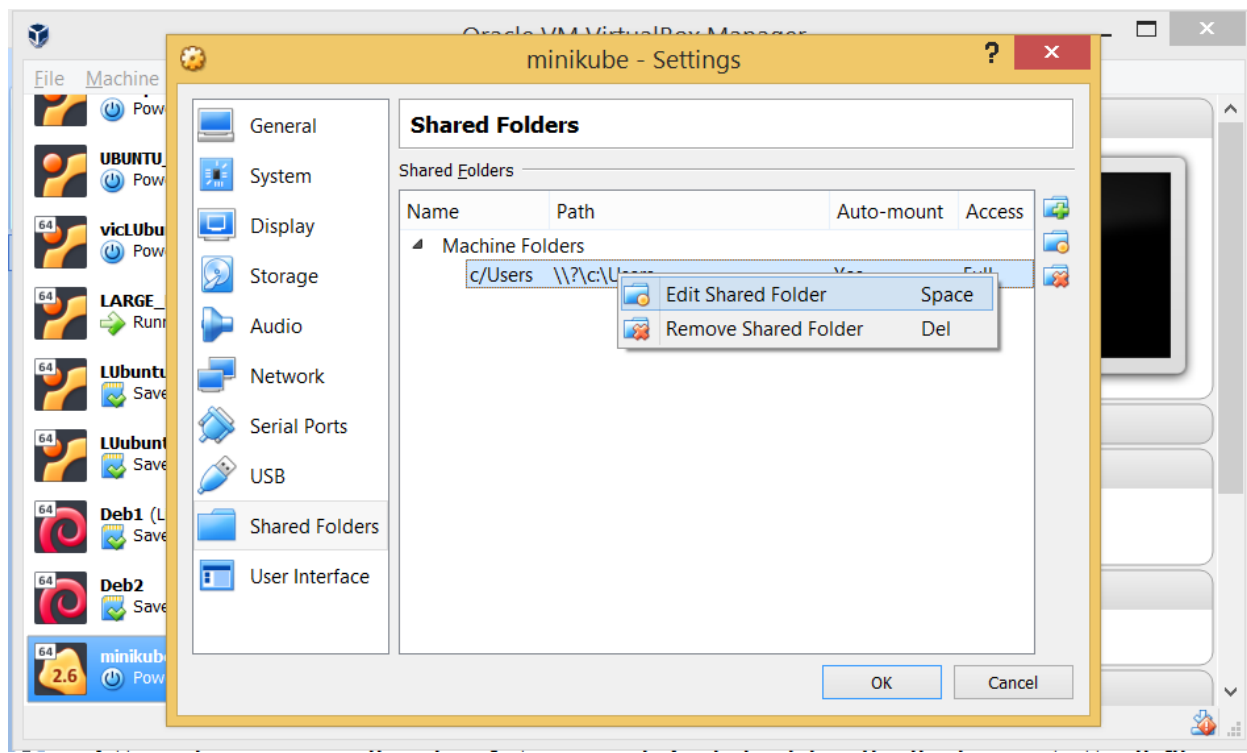
Selezionare poi la voce di menù a sinistra "Shared Folders" per abilitare un una directory condivisa con host.



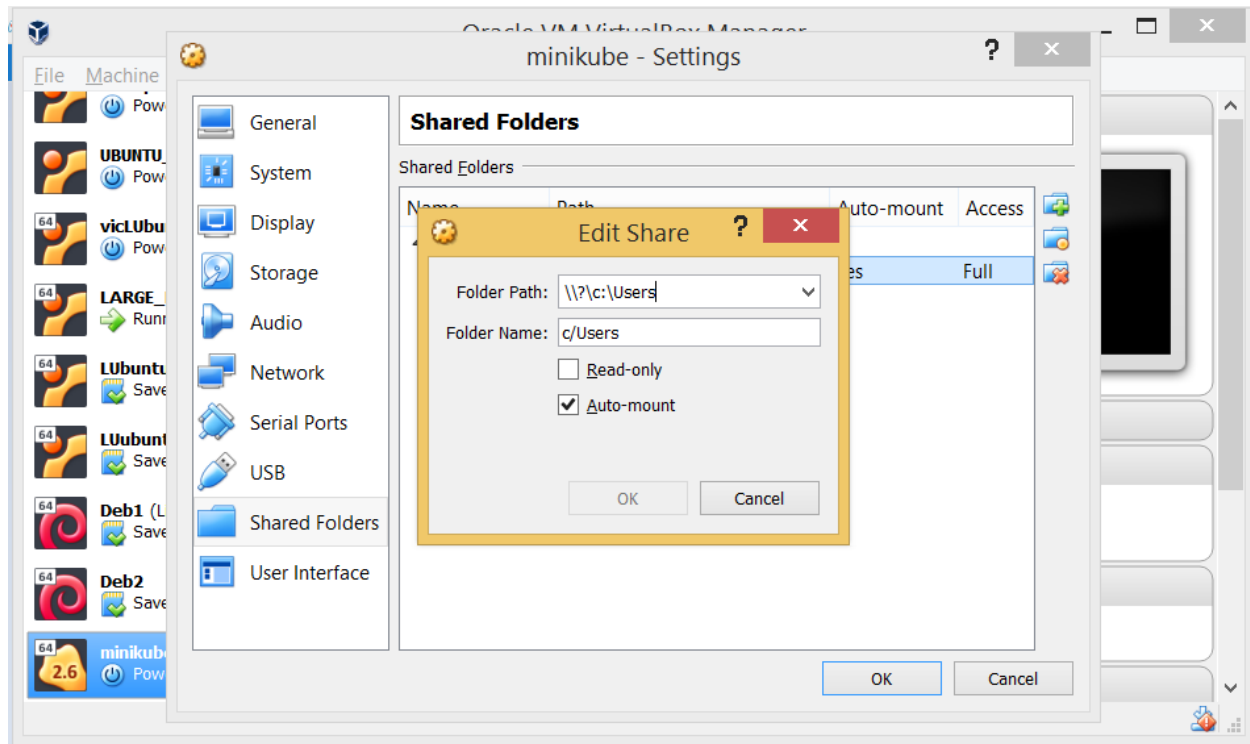
Nella finestra Shared Folders che si apre, potete vedere, sotto Machine Folder, l'elenco delle directory condivise. Vedrete che è già configurato un folder condiviso che, nella macchina virtuale è visibile come se fosse il percorso /c/Users/ mentre nella macchina Windows ospitante corrisponde alla directory C:\Users\



Per vedere meglio questo dettaglio, selezionare il folder c/Users, cliccare col pulsante destro del mouse e, nel menù che si apre, selezionare Edit Shared Folder.



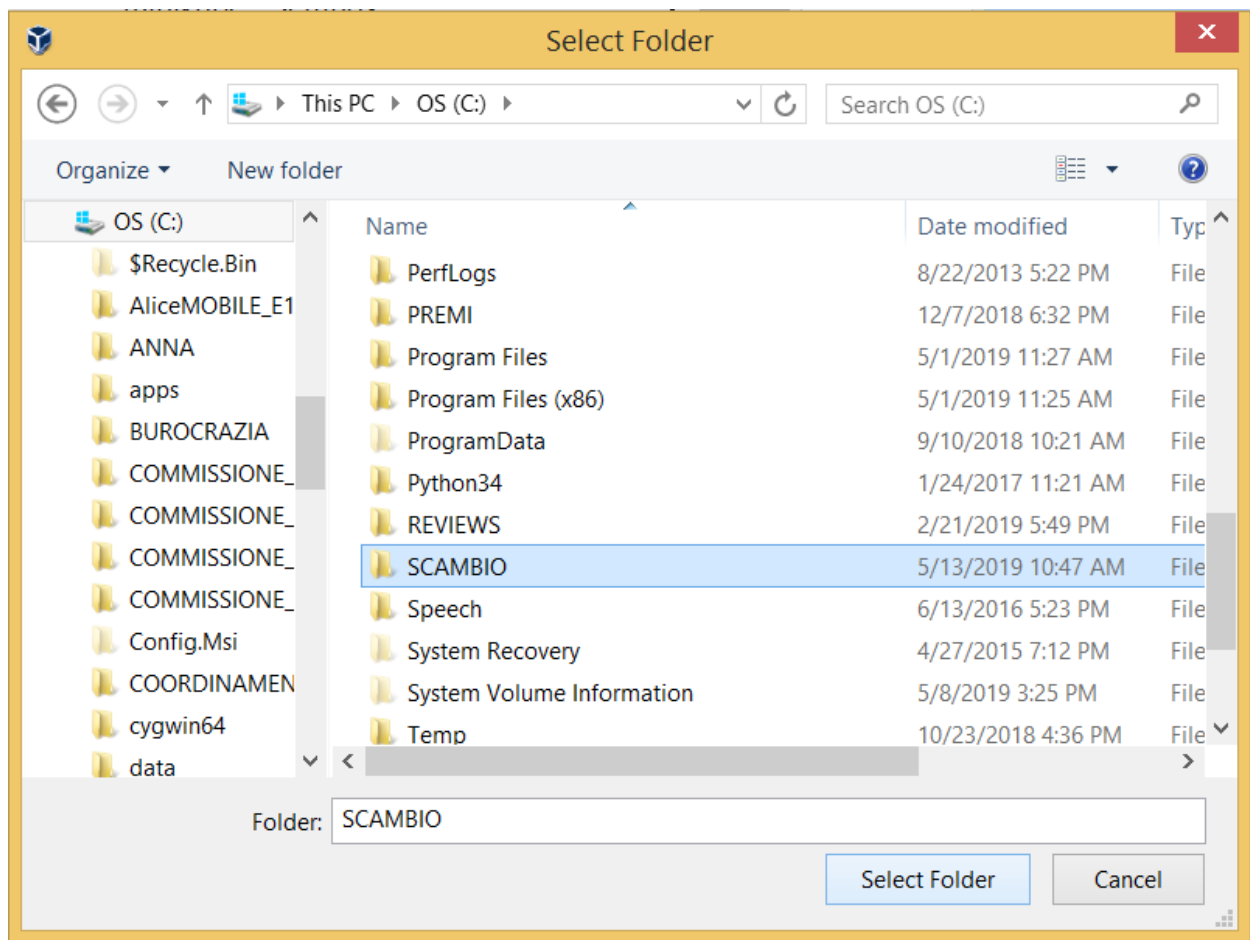
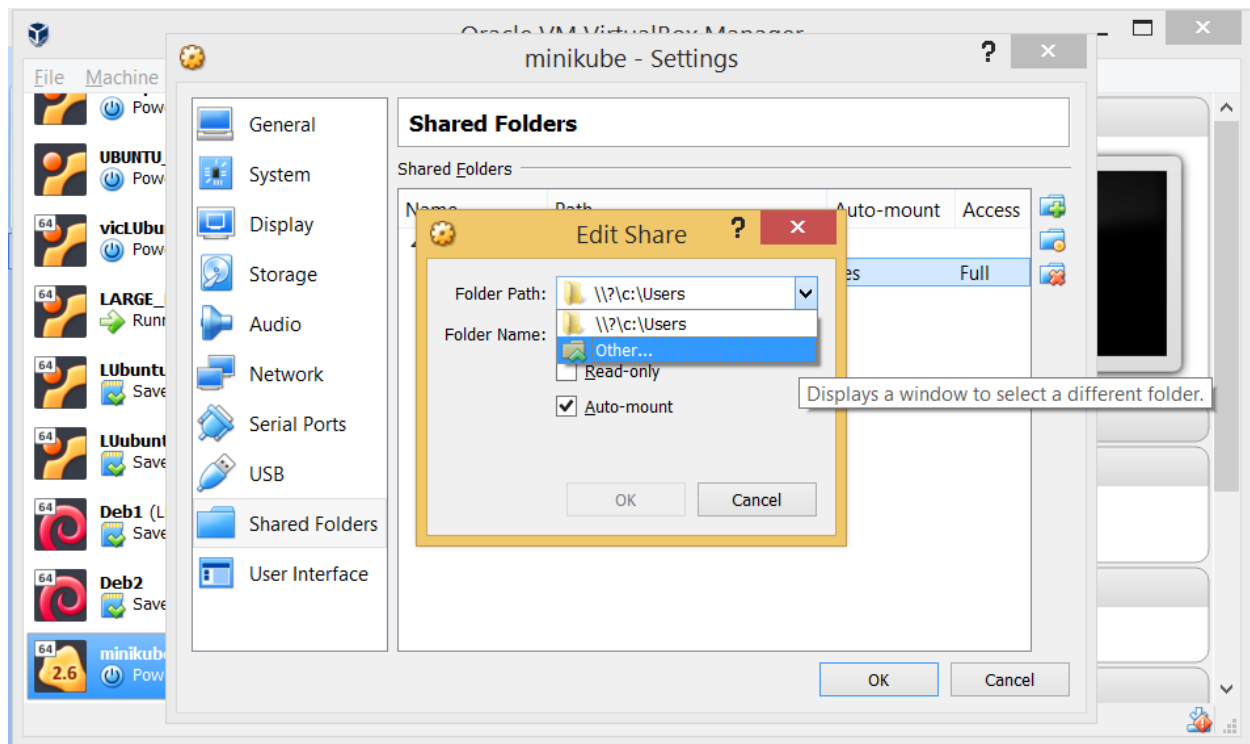
Si apre questa nuova finestra qui sotto, che mette in evidenza come il Folder Name (come lo vede la macchina ospitata) sia "c/Users", mentre sulla macchina ospitante la directory condivisa parte dal percorso **Folder Path** `\\?\c:\Users` ovvero le directory con le home degli utenti di windows.

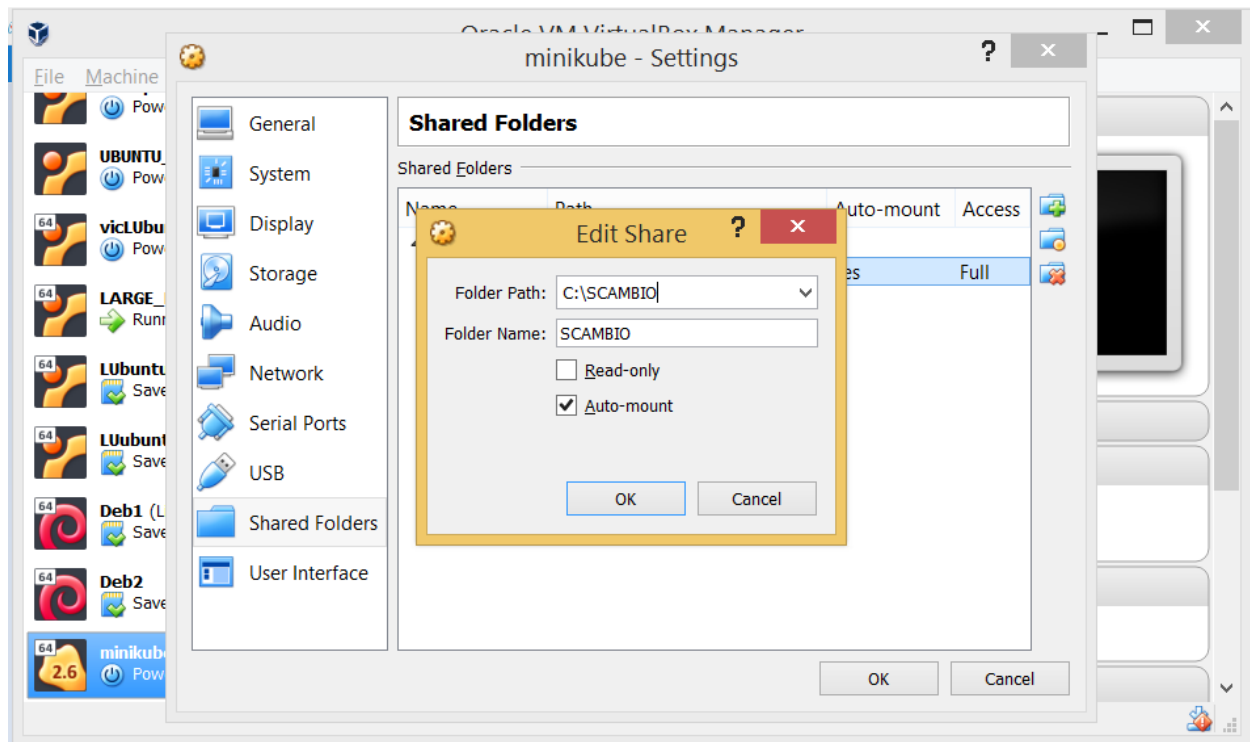


Quando userete la directory condivisa, attenzione a quello che fate, perché si rischia di eliminare tutto il file system di Windows.

Suggerisco di modificare la condivisione, limitando la condivisione ad una sola directory del file system di windows, C:\SCAMBIO

Per fare questo, modificate il campo **Folder Path**, inserendo il nuovo percorso di una directory esistente in windows, ad esempio **C:\SCAMBIO** e cambiando il Folder name in "SCAMBIO" seguendo l'esempio delle immagini qui sotto.



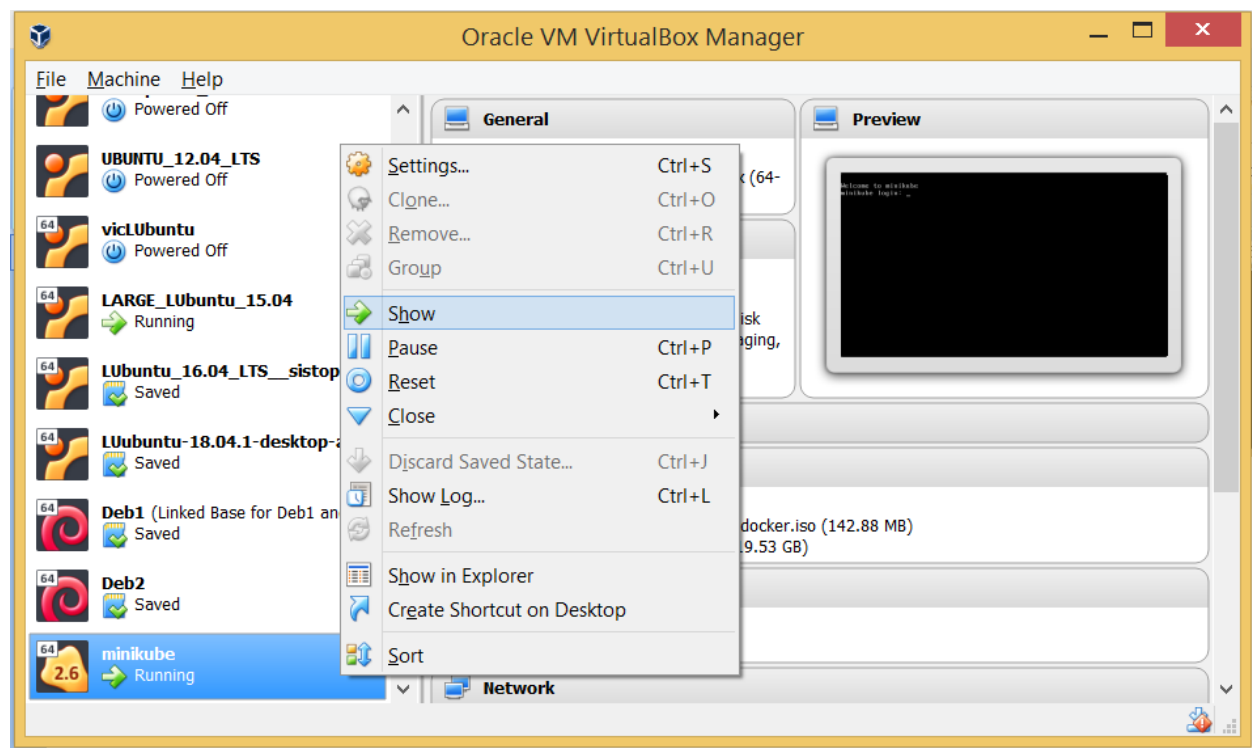


SELEZIONARE più volte ok per tornare alla finestra principale.

Se ora andate nella shell **cygwin** di windows e fate partire nuovamente il cluster con "**minicube start**", poi nella finestra di virtualbox selezionate la macchina **minicube**, cliccate col pulsante destro, cliccate su **show**, si apre una finestra di login, entrate con username root e senza password, ed eseguite il comando `ls -alhR /SCAMBIO` allora vedrete il contenuto della directory C:\SCAMBIO su windows.

```
"minikube" stopped.

vittorio@alien ~
$ minikube start
minikube v1.0.1 on windows (amd64)
$ Downloading Kubernetes v1.14.1 images in the background ...
Tip: Use 'minikube start -p <name>' to create a new cluster, or '
minikube delete' to delete this one.
: Restarting existing virtualbox VM for "minikube" ...
: Waiting for SSH access ...
: "minikube" IP address is 192.168.99.100
: Configuring Docker as the container runtime ...
: Version of container runtime is 18.06.3-ce
: Waiting for image downloads to complete ...
: Preparing Kubernetes environment ...
: Pulling images required by Kubernetes v1.14.1 ...
: Relaunching Kubernetes v1.14.1 using kubeadm ...
: Waiting for pods: apiserver proxy etcd scheduler controller dns
: Updating kube-proxy configuration ...
: Verifying component health .....
+ kubect! is now configured to use "minikube"
= Done! Thank you for using minikube!
```





```
minikube [Running] - Oracle VM VirtualBox

Welcome to minikube
minikube login: root

      _
     _/
    _/
   _/
  _/
 _/
/_/

# ls /
SCAMBIO data etc init lib64 media opt root sbin sys usr
bin dev home lib linuxrc mnt proc run srv tmp var
# ls /SCAMBIO/README_VIC.txt
/SCAMBIO/README_VIC.txt
# _
```

g) Usare la dashboard di kubernetes.

Per usare la dashboard di controllo di kubernetes, all'interno della finestra cygwin eseguire il comando:

minikube dashboard

Quick Start

Start a cluster by running:
`minikube start`

Access Kubernetes Dashboard within Minikube:
`minikube dashboard`

Once started, you can interact with your cluster using `kubectl`, just like any other Kubernetes cluster. For instance, starting a server:
`kubectl run hello-minikube --image=k8s.gcr.io/echoserver:1.4 --port=8080`

Exposing a service as a NodePort
`kubectl expose deployment hello-minikube --type=NodePort`

minikube makes it easy to open this exposed endpoint in your browser:
`minikube service hello-minikube`

Start a second local cluster:
`minikube start -p cluster2`

Stop your local cluster:
`minikube stop`

Delete your local cluster:
`minikube delete`